

Numpy

October 6, 2025

1 Numpy

[Numpy or Numerical Python](#) is a python package for scientific computing. It is impossible to miss this library when we are working in the field of machine learning. Before we get into detail about Numpy and its features we need to understand some basic concept about array.

1.1 Arrays

Array in programming is the order series of homogeneous(similar type) data. In python we have `list` and `tuples` which work as an array. Since python is dynamically typed language using these data structure comes with high cost of space and time while performing heavy mathematical operations like matrix multiplication which is at the heart of linear algebra.

1.1.1 Dynamic Vs. Static Typing

In programming languages like C,C++,Java etc. we have to declare variable type explicitly but in dynamically typed programming language like python we do not need to specify the data type of the variable. Let us take an example of script that will add two numbers.

```
#include<stdio.h>
void main(){
    int a=10;
    int b=20;
    int c;
    c=a+b;
    printf("The sum of %d and %d is %d",a,b,c);
}
```

This same program can be written in python as

```
a=10
b=20
c=a+b
print("sum of {0} and {1} is {2}".format(a,b,c))
```

In C, we must declare the data type of every variable (like `int a, b, c;`). In Python, we don't need to declare types — we can just assign values directly. This makes Python programs quicker and easier to write. However, it also affects **performance** and **memory usage**. Since Python doesn't know the data type in advance, it can't optimize the program as well as C. It also has to use more memory to store extra information (metadata) about each variable.

The data structure used in python that can be mutated (changed) is list. We can create a list as

```
[1]: a = [1,2,3,4,5]
```

```
[2]: type(a[0])
```

```
[2]: int
```

```
[3]: len(a)
```

```
[3]: 5
```

Each element in the list is given an index (position value) which starts from 0 and goes to **number of elements in list**. In our case there are 5 elements in the list so the indexing goes from 0 to 4. The index of the first element is 0. If we try to get the element using the index that does not exist, we will get an error.

```
[4]: a[5]
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[4], line 1  
----> 1 a[5]  
  
IndexError: list index out of range
```

Since python is a dynamically typed language we can have heterogeneous lists

```
[5]: b=[1, 'two', False, 10.5]
```

```
[6]: for i in range(len(b)):  
      print(type(b[i]))
```

```
<class 'int'>  
<class 'str'>  
<class 'bool'>  
<class 'float'>
```

This flexibility has a downside. To support different types, each element in a Python list stores extra details like its type information, reference count, and other metadata—essentially, each element is a full Python object. When all elements are of the same type, this extra information becomes unnecessary. In such cases, storing data in a fixed-type array is much more efficient in terms of both memory and speed.

Numpy provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more. [Source Numpy Documentation](#)

1.2 Getting Started with Numpy

Numpy is very easy to install once python is installed. Please follow the instructions from [Official Numpy Documentation](#). We can use the pip command to install the numpy package.

The Numpy package is imported as `np` this is not necessary but it a become a common practice.

```
[7]: !pip install numpy
```

```
Requirement already satisfied: numpy in /media/xenon/New  
Volume/broadway/DataScience_8AM/venv/lib/python3.12/site-packages (2.3.3)
```

```
[8]: import numpy as np
```

Lets check the version

```
[9]: np.__version__
```

```
[9]: '2.3.3'
```

We can use `np.array` method in the numpy package to create an array. It converts Python sequences (like lists, tuples, or nested lists) into NumPy arrays. It inspects the input to determine the shape and data type. If all elements are of the same type and length, it creates a regular numeric array; otherwise, it may fall back to an object array.

Let us look at some examples

```
[10]: arr1 = np.array([1, 2, 3])           # 1D array  
      arr2 = np.array([[1, 2], [3, 4]])    # 2D array  
      arr3 = np.array((5, 6, 7))          # From tuple
```

```
[11]: arr1
```

```
[11]: array([1, 2, 3])
```

```
[12]: arr2
```

```
[12]: array([[1, 2],  
            [3, 4]])
```

```
[13]: arr3
```

```
[13]: array([5, 6, 7])
```

Let's check the data types of the array

```
[14]: type(arr1)
```

```
[14]: numpy.ndarray
```

```
[15]: arr1.dtype
```

```
[15]: dtype('int64')
```

```
[16]: type(arr2)
```

```
[16]: numpy.ndarray
```

```
[17]: arr2.dtype
```

```
[17]: dtype('int64')
```

```
[18]: type(arr3)
```

```
[18]: numpy.ndarray
```

```
[19]: arr3.dtype
```

```
[19]: dtype('int64')
```

Creating an array from a heterogeneous python list.

```
[20]: arr4 = np.array([1, 'one', True, 4, 5])
```

```
[21]: arr4.dtype
```

```
[21]: dtype('<U21')
```

If we check the array we can see that all the elements are converted to string

```
[22]: arr4
```

```
[22]: array(['1', 'one', 'True', '4', '5'], dtype='<U21')
```

```
[23]: for item in arr4:
      print(type(item))
```

```
<class 'numpy.str_'>
<class 'numpy.str_'>
<class 'numpy.str_'>
<class 'numpy.str_'>
<class 'numpy.str_'>
```

We can use the `dtype` parameter to create an numpy array of a specific data type. We can refer to the numpy array [dtypes documentation](#) for details on the data types we can use.

```
[28]: np.array([10,20,30,45],dtype='float32')
```

```
[28]: array([10., 20., 30., 45.], dtype=float32)
```

1.2.1 Dimension of Array

An array can also contain array as its element. The number of array an variable holds can be considered as its dimension. 1-D array is just a ordered list of similar data. 2-D array is an array of array. 3-D array is an array that contains two arrays as its element. A N-D array is an array that contains N number of arrays as its element. We can visulize a 1-D array as a row.

<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>	<u>9</u>
----------	----------	----------	----------	----------	----------	----------	----------	----------	----------

The figure above is a visulization is an 1 dimesional (1-D) array. A 2-D array can be visulized as a table. The figure below shows a 3X3 array. It means that each item in the array contains an array of 3.

[0,0]	[0,1]	[0,2]
[1,0]	[1,1]	[1,2]
[2,0]	[2,1]	[2,2]

The **number of dimensions** refers to how many axes an array has: - A 1D array has one axis (like a simple list). - A 2D array has two axes (rows and columns, like a matrix). - A 3D array has three axes (depth, rows, columns).

We can check the number of dimensions using the `.ndim` attribute.

```
[33]: arr1 = np.array([1, 2, 3])           # 1D array
      arr2 = np.array([[1, 2], [3, 4]])    # 2D array
      arr3 = np.array([[[1], [2]], [[3], [4]]]) # 3D array

      print("arr1 ndim:", arr1.ndim)
```

```
print("arr2 ndim:", arr2.ndim)
print("arr3 ndim:", arr3.ndim)
```

```
arr1 ndim: 1
arr2 ndim: 2
arr3 ndim: 3
```

The **shape** of an array is a tuple that shows the number of elements along each dimension. For example: - A shape of (3,) means the array has 3 elements in one dimension. - A shape of (2, 2) means 2 rows and 2 columns. - A shape of (2, 2, 1) means 2 blocks, each with 2 rows and 1 column.

We can check the shape using the `.shape` attribute.

```
[34]: print("arr1 shape:", arr1.shape)
      print("arr2 shape:", arr2.shape)
      print("arr3 shape:", arr3.shape)
```

```
arr1 shape: (3,)
arr2 shape: (2, 2)
arr3 shape: (2, 2, 1)
```

The **size** of an array is the **total number of elements** it contains. This is simply the product of the shape values.

For example, if the shape is (2, 3), then the size is $2 \times 3 = 6$ elements.

We can check the total number of elements using the `.size` attribute.

```
[35]: print("arr1 size:", arr1.size)
      print("arr2 size:", arr2.size)
      print("arr3 size:", arr3.size)
```

```
arr1 size: 3
arr2 size: 4
arr3 size: 4
```

The **shape** of an array is a tuple that shows the number of elements along each dimension. For example: - A shape of (3,) means the array has 3 elements in one dimension. - A shape of (2, 2) means 2 rows and 2 columns. - A shape of (2, 2, 1) means 2 blocks, each with 2 rows and 1 column.

We can check the shape using the `.shape` attribute.

```
[ ]:
```

In Numpy there are multiple ways to create different types of arrays. Now we will take a look at some of them

An integer array with 20 zeros.

```
[36]: np.zeros(10, dtype='int')
```

```
[36]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

We can also create an multidimesional array by passing the size of an array. The size of an array can be thought as a matrix. A 3X5 array means that a matrix with 3 rows and 5 columns.

```
[37]: x=np.zeros((3,5),dtype='int')
      print(x.shape)
      x
```

```
(3, 5)
```

```
[37]: array([[0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0]])
```

An Array with 10 ones

```
[38]: np.ones(10,dtype='int32')
```

```
[38]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=int32)
```

Random Values An array that contains uniformly distributed random numbers between 0 and 1.

```
[39]: np.random.random(10)
```

```
[39]: array([0.21065156, 0.47566054, 0.42209566, 0.79348544, 0.68040116,
            0.06387248, 0.01177256, 0.35387248, 0.31141596, 0.39738471])
```

```
[40]: np.random.random((10,10))
```

```
[40]: array([[0.61401967, 0.78223659, 0.92698405, 0.69299535, 0.64610361,
            0.42998647, 0.24322618, 0.97734433, 0.45432866, 0.96752312],
            [0.36595456, 0.03849385, 0.15486139, 0.7075914 , 0.4185943 ,
            0.55509839, 0.84073355, 0.44771419, 0.95893737, 0.40281516],
            [0.81094224, 0.26601101, 0.86869159, 0.79077598, 0.99802021,
            0.7528659 , 0.05937604, 0.50861066, 0.30424593, 0.86677635],
            [0.40989689, 0.50829863, 0.28597018, 0.7978629 , 0.96738039,
            0.5282052 , 0.95555143, 0.29593828, 0.61875775, 0.11505974],
            [0.61687459, 0.3990489 , 0.80814799, 0.61632667, 0.23945642,
            0.30642095, 0.62510264, 0.05601324, 0.77225338, 0.91636197],
            [0.65090127, 0.01169745, 0.2001487 , 0.11386165, 0.41806824,
            0.62328953, 0.68069072, 0.55709465, 0.11601101, 0.3260804 ],
            [0.57118294, 0.55602814, 0.33386017, 0.03448299, 0.93540243,
            0.49745531, 0.5281231 , 0.42242134, 0.00793761, 0.12325704],
            [0.53969711, 0.21114983, 0.4137512 , 0.27685152, 0.27813214,
            0.0240371 , 0.26755615, 0.66613175, 0.35795957, 0.033192 ],
            [0.4493867 , 0.94771815, 0.65349614, 0.18606649, 0.17923766,
```

```
0.72760348, 0.75841071, 0.59289767, 0.54775193, 0.28639122],
[0.17960928, 0.80849619, 0.35295147, 0.7597363 , 0.29694865,
0.62789707, 0.48151081, 0.63250819, 0.43417227, 0.07189218]])
```

An array of integers that contains random values in interval [1,101) it means that 1 is inclusive and 101 is exclusive. We will get an array of that contains values between 1 and 100

```
[41]: np.random.randint(1,101,(10,10))
```

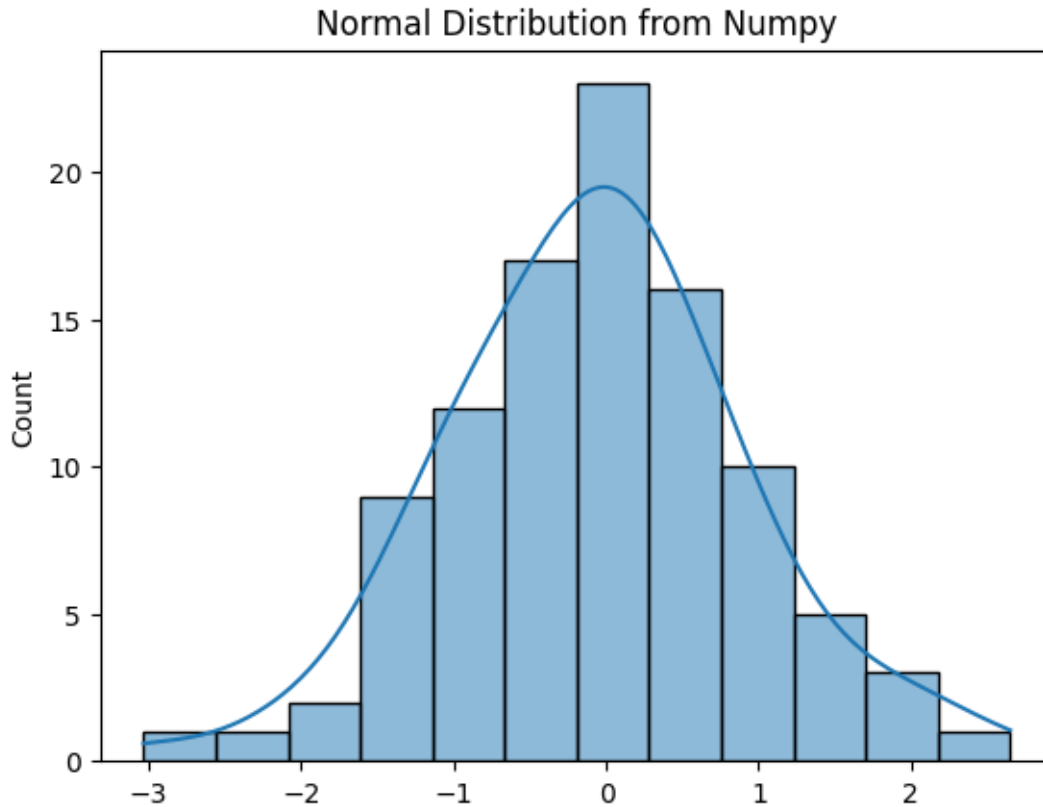
```
[41]: array([[90, 75, 65, 10, 96, 26, 59, 44, 62,  4],
           [32,  9, 76, 89, 87, 96, 73, 92, 22, 67],
           [28, 17, 41, 51, 95,  9, 70, 90, 67, 28],
           [65, 49, 49, 22, 88, 52, 94, 75, 57, 25],
           [89, 87, 58, 24, 75,  5, 46, 86, 83, 60],
           [57, 38, 48, 85, 30, 45, 20, 43, 79, 86],
           [ 8, 62, 50, 46, 11,  3, 29,  7, 78, 80],
           [41, 47, 43, 13, 24, 75, 10, 69,  9, 56],
           [10, 20, 85, 33, 25, 73, 56, 75, 95, 25],
           [98, 33, 32, 24, 70, 56,  2, 40, 73,  9]])
```

An Array with normally distributed random values

```
[42]: norm = np.random.normal(0,1,100)
```

```
[45]: import matplotlib.pyplot as plt
import seaborn as sns
sns.histplot(norm,kde=True)
plt.title('Normal Distribution from Numpy')
```

```
[45]: Text(0.5, 1.0, 'Normal Distribution from Numpy')
```

Arrays from Range An Array of numbers between 0 and 100 which are separated by three

```
[46]: np.arange(0, 100, 3)
```

```
[46]: array([ 0,  3,  6,  9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48,
          51, 54, 57, 60, 63, 66, 69, 72, 75, 78, 81, 84, 87, 90, 93, 96, 99])
```

An array of 100 elements evenly spaced between 0 and 1

```
[47]: np.linspace(0,1,100)
```

```
[47]: array([0.          , 0.01010101, 0.02020202, 0.03030303, 0.04040404,
          0.05050505, 0.06060606, 0.07070707, 0.08080808, 0.09090909,
          0.1010101 , 0.11111111, 0.12121212, 0.13131313, 0.14141414,
          0.15151515, 0.16161616, 0.17171717, 0.18181818, 0.19191919,
          0.2020202 , 0.21212121, 0.22222222, 0.23232323, 0.24242424,
          0.25252525, 0.26262626, 0.27272727, 0.28282828, 0.29292929,
          0.3030303 , 0.31313131, 0.32323232, 0.33333333, 0.34343434,
          0.35353535, 0.36363636, 0.37373737, 0.38383838, 0.39393939,
          0.4040404 , 0.41414141, 0.42424242, 0.43434343, 0.44444444,
```

```

0.45454545, 0.46464646, 0.47474747, 0.48484848, 0.49494949,
0.50505051, 0.51515152, 0.52525253, 0.53535354, 0.54545455,
0.55555556, 0.56565657, 0.57575758, 0.58585859, 0.5959596 ,
0.60606061, 0.61616162, 0.62626263, 0.63636364, 0.64646465,
0.65656566, 0.66666667, 0.67676768, 0.68686869, 0.6969697 ,
0.70707071, 0.71717172, 0.72727273, 0.73737374, 0.74747475,
0.75757576, 0.76767677, 0.77777778, 0.78787879, 0.7979798 ,
0.80808081, 0.81818182, 0.82828283, 0.83838384, 0.84848485,
0.85858586, 0.86868687, 0.87878788, 0.88888889, 0.8989899 ,
0.90909091, 0.91919192, 0.92929293, 0.93939394, 0.94949495,
0.95959596, 0.96969697, 0.97979798, 0.98989899, 1.      ])
```

Matrix We can use Numpy for matrix calculation as well. Lets see soem of the examples of how we can intilize different types of Matrix using Numpy.

Identity Matrix (Matrix with 1 in main diagonal and other element 0)

```
[48]: np.eye(5,5,dtype='int32')
```

```
[48]: array([[1, 0, 0, 0, 0],
            [0, 1, 0, 0, 0],
            [0, 0, 1, 0, 0],
            [0, 0, 0, 1, 0],
            [0, 0, 0, 0, 1]], dtype=int32)
```

A simple 3X3 matrix

```
[49]: np.array([[4,7,6],[1,2,5],[9,3,8]])
```

```
[49]: array([[4, 7, 6],
            [1, 2, 5],
            [9, 3, 8]])
```

```
[50]: a = np.matrix('1 2; 3 4')
```

```
[51]: a
```

```
[51]: matrix([[1, 2],
            [3, 4]])
```

1.3 Read and Write From File

Lets say we have a text file that has following data

column_1	column_2	column_3
0.894640452	0.869125722	0.941058791
0.309549961	0.509928931	0.051745242
0.250838896	0.168732778	0.519133912
0.559179939	0.694110118	0.124523412

column_1	column_2	column_3
0.987030624	0.321504897	0.532219472
0.347208744	0.278191203	0.55857671
0.245945774	0.128405281	0.755943007
0.160124686	0.098261041	0.017169905
0.904910103	0.717794888	0.786813136

```
[52]: x, y, z = np.loadtxt('data/data.txt',
                        skiprows=1,
                        unpack=True)
```

The parameter `unpack=True` is given so that each column is assigned to different variable. `x` is assigned to first column and so on.

```
[53]: x
```

```
[53]: array([0.89464045, 0.30954996, 0.2508389 , 0.55917994, 0.98703062,
           0.34720874, 0.24594577, 0.16012469, 0.9049101 ])
```

We can save the array in a file as

```
[54]: np.savetxt('data/out.txt', x, delimiter=',')
```

1.4 Vectorization

Vectorization a style of computer programming where operations are applied to whole arrays instead of individual elements. Lets take an example using both pure python and numpy where we add each element of a array with corresponding element of same array.

```
[55]: def element_wise_add_python(arr1,arr2):
        return [arr1+arr2 for (arr1,arr2) in zip(arr1,arr2)]
```

We can use numpy to perform vector operation or to perform vectorization.

```
[56]: def element_wise_add_numpy(arr1,arr2):
        return np.add(arr1,arr2)
```

Lets use Simple benchmarking in a large array so that we can see the difference

```
[57]: arr1 = np.random.randint(1,10000,50000)
        arr2 = np.random.randint(50,10000,50000)
```

```
[58]: %%timeit
        X=element_wise_add_python(arr1,arr2)
```

2.61 ms $\pm$ 63.6 s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

```
[59]: %%timeit
        y=element_wise_add_numpy(arr1,arr2)
```

10 s $\pm$ 204 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

From the above result we can see how we can increase the performance by performing vectorization. Per operation the pure python method took around 16.3 micro second and the numpy took 31.9 nano second per operation which is much faster than pure python implementation.

We can perform various operations to an array. We are going to discuss some of them in this tutorial.

```
[60]: arr= np.array([-2, -1, 0, 1, 2])
```

Get Absolute Value

```
[61]: abs(arr)
```

```
[61]: array([2, 1, 0, 1, 2])
```

Apply Trigonometric Operation

```
[62]: np.sin(arr)
```

```
[62]: array([-0.90929743, -0.84147098,  0.          ,  0.84147098,  0.90929743])
```

```
[63]: np.tan(arr)
```

```
[63]: array([ 2.18503986, -1.55740772,  0.          ,  1.55740772, -2.18503986])
```

Exponentials and Logarithms

```
[64]: np.exp(arr)
```

```
[64]: array([0.13533528, 0.36787944, 1.          , 2.71828183, 7.3890561 ])
```

```
[65]: arr1=np.array([1,2,3,4,5,6,7,8,9,12])
```

```
[66]: np.log(arr1)
```

```
[66]: array([0.          , 0.69314718, 1.09861229, 1.38629436, 1.60943791,
          1.79175947, 1.94591015, 2.07944154, 2.19722458, 2.48490665])
```

```
[67]: # Log Base 2
      np.log2(arr1)
```

```
[67]: array([0.          , 1.          , 1.5849625 , 2.          , 2.32192809,
          2.5849625 , 2.80735492, 3.          , 3.169925  , 3.5849625 ])
```

```
[68]: # Log Base 10
      np.log10(arr1)
```

```
[68]: array([0.          , 0.30103   , 0.47712125, 0.60205999, 0.69897   ,
          0.77815125, 0.84509804, 0.90308999, 0.95424251, 1.07918125])
```

Min and Max Value

```
[69]: np.min(arr1)
```

```
[69]: np.int64(1)
```

```
[70]: np.max(arr1)
```

```
[70]: np.int64(12)
```

Some Basic Mathematical Operations

```
[71]: arr1=[1,2,3,4,5]  
      arr2=[6,7,8,9,10]
```

```
[72]: np.add(arr1,arr2)
```

```
[72]: array([ 7,  9, 11, 13, 15])
```

```
[73]: np.subtract(arr2,arr1)
```

```
[73]: array([5, 5, 5, 5, 5])
```

```
[74]: np.multiply(arr1,arr2)
```

```
[74]: array([ 6, 14, 24, 36, 50])
```

1.5 Matrix Operations in Numpy

At the heart of linear algebra is matrix operations. Linear algebra is the backbone of machine learning algorithm so for task like machine learning we need to perform various kind of matrix operations. These matrix operations can be expensive in pure python implementation. We can use Numpy to perform such operations which make these tasks faster and easier.

1.5.1 Transpose of a matrix

We can transpose a matrix by converting rows into columns and columns into rows.

```
[75]: import numpy as np  
  
      arr1 = np.array([[1, 2, 3], [4, 5, 6]])
```

```
[76]: arr1
```

```
[76]: array([[1, 2, 3],  
          [4, 5, 6]])
```

```
[77]: arr1_transpose = arr1.transpose()
```

```
[78]: arr1_transpose
```

```
[78]: array([[1, 4],
           [2, 5],
           [3, 6]])
```

The arr1 was original marix. We used `transpose()` method to transpose this matrix

1.5.2 Inverse of a Matrix

```
[79]: x = np.array([[1,2],[3,4]])
      x_inv = np.linalg.inv(x)
```

```
[80]: print(x)
```

```
[[1 2]
 [3 4]]
```

```
[81]: print(x_inv)
```

```
[[ -2.   1. ]
 [ 1.5 -0.5]]
```

1.5.3 Dot Product of Vectors

Given the two vectors $a = (a_1, a_2, a_3 \dots a_n)$ and $b = (b_1, b_2, b_3, \dots b_n)$ then the dot product is given by

$$a.b = a_1 * b_1 + a_2 * b_2 + \dots + a_n * b_n$$

```
[82]: x=np.array([1,2,3])
```

```
[83]: y=np.array([4,5,6])
```

The result of dot product of x and y should be

```
[84]: 1*4+2*5+3*6
```

```
[84]: 32
```

Using Numpy

```
[85]: np.dot(x,y)
```

```
[85]: np.int64(32)
```

1.6 Cosine Simalirity Example

The Angle between two vectors is given by

$$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

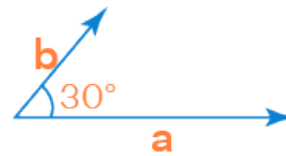
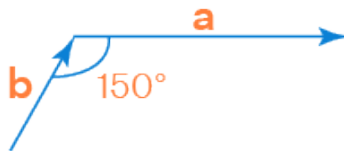
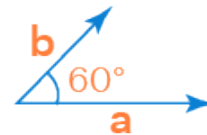
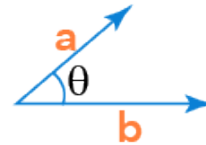
Angle Between Two Vectors



Not Tail - Tail



Tail - Tail



```
[102]: import numpy as np
```

```
[103]: # User-Movie rating matrix (rows = users, columns = movies)
ratings = np.array([
    [5, 4, 0, 1], # User 1
    [4, 0, 0, 1], # User 2
    [1, 1, 0, 5], # User 3
    [0, 0, 5, 4]  # User 4
])
```

```
[104]: # Step 1: Compute dot product between all users
dot_product = ratings @ ratings.T # shape (4,4)
```

```
[105]: dot_product
```

```
[105]: array([[42, 21, 14,  4],
            [21, 17,  9,  4],
            [14,  9, 27, 20],
```

```
[ 4,  4, 20, 41]])
```

```
[106]: # Step 2: Compute norm (magnitude) of each user's vector
norms = np.linalg.norm(ratings, axis=1)
```

```
[108]: norms
```

```
[108]: array([6.4807407 , 4.12310563, 5.19615242, 6.40312424])
```

```
[111]: # Step 4: Cosine similarity matrix
cosine_similarity = dot_product / (norms[:, None] * norms[None, :])
```

```
[112]: print(np.round(cosine_similarity, 2))
```

```
[[1.    0.79 0.42 0.1 ]
 [0.79  1.    0.42 0.15]
 [0.42  0.42  1.    0.6 ]
 [0.1   0.15 0.6   1.   ]]
```

This shows, for example, User 1 is most similar to User 2, less to User 3, and almost no similarity with User 4.

1.7 Conclusion

In this tutorial we learned about Numpy package. We explored various operations we can perform on array created using Numpy.